🟦 Python OOP — Level 1, 2 & 3

🟢 Beginner → 🟡 Intermediate → 🔴 Advanced

📘 Quick Revision + Interview + Exam Notes

🟢 LEVEL 1 — BEGINNER

🧱 Foundation: Classes, Objects, Attributes, Methods, `self`, `__init__`

---

1. Why OOP?

OOP = **Object-Oriented Programming**.

It organizes related **data + behavior** into objects.

Without OOP:

```
task_name = "Learn OOP"
task_status = "Pending"
task_priority = "High"
```

With OOP:

```
class Task:
    ...
```

Now one `Task` object can contain:

```
Task
├── name
├── status
└── priority
```

🌍 Real-world idea

Real world	Python OOP
Blueprint	Class
Actual house	Object
House color	Attribute

Open door	Method

---

2. Class

A **class** is a blueprint/template for creating objects.

```
class Student:
    pass
```

Common naming convention:

```
class Student:
class TaskManager:
class FileOrganizer:
```

Use **PascalCase** for class names.

⭐ **Most forgettable mistake:** Class names should normally use **PascalCase**, not lowercase or snake_case.

---

3. Object / Instance

An object is an actual instance created from a class.

```
class Student:
    pass

student1 = Student()
student2 = Student()
```

Relationship:

```
Student
   ↓
Blueprint

Student()
   ↓
Object
```

`student1` and `student2` are separate objects/instances.

⭐ **Most forgettable mistake:** A class is only a blueprint; the object is the actual created instance.

---

4. Attributes

Attributes store object data.

```
student1.name = "Khushi"
student1.age = 21
```

Now:

```
student1
├── name = Khushi
└── age = 21
```

Access:

```
print(student1.name)
print(student1.age)
```

---

5. `__init__()`

`__init__()` initializes an object when it is created.

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Create:

```
student = Student("Khushi", 21)
```

Conceptually:

```
Student()
   ↓
object created
   ↓
__init__() runs
   ↓
object initialized
```

⭐ **Most forgettable mistake:** `__init__()` initializes the object; it is not technically responsible for creating the object itself.

---

6. `self`

`self` refers to the **current object/instance**.

```
class Student:

    def __init__(self, name):
        self.name = name
```

For:

```
student1 = Student("Khushi")
```

`self` refers to `student1`.

For:

```
student2 = Student("Alex")
```

`self` refers to `student2`.

Therefore:

```
self.name
```

means:

This particular object's `name`.

⭐ **Most forgettable mistake:** `self` is not a normal variable containing the name; it refers to the current object.

---

7. Parameter vs Attribute

Very important:

```
class Student:

    def __init__(self, name):
        self.name = name
```

Code	Meaning
`name`	parameter/local value
`self.name`	attribute stored in object
`self.name = name`	store received value in object

Think:

```
name
 ↓
temporary input
```

```
self.name
  ↓
object's stored data
```

⭐ **Most forgettable mistake:** `name = name` does not store data inside the object; `self.name = name` does.

---

8. Methods

A function inside a class is called a **method**.

```
class Student:

    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f"Hello, my name is {self.name}")
```

Use:

```
student = Student("Khushi")
student.introduce()
```

Output:

```
Hello, my name is Khushi
```

---

9. Attribute vs Method

Attribute	Method
Stores data	Performs action
`student.name`	`student.introduce()`
Usually no `()`	Usually uses `()`

Example:

```
student.name
student.introduce()
```

⭐ **Most forgettable mistake:** Attributes are accessed without `()`, while methods are normally called with `()`.

---

10. First Real OOP Example

```
class Task:

    def __init__(self, name, priority):
        self.name = name
        self.priority = priority
        self.status = "Pending"

    def complete(self):
        self.status = "Completed"
```

Usage:

```
task1 = Task("Learn OOP", "High")

print(task1.status)

task1.complete()

print(task1.status)
```

Output:

```
Pending
Completed
```

💡 Core OOP idea:

```
Object
├── Data
│    ├── name
│    ├── priority
│    └── status
│
└── Behavior
     └── complete()
```

🧠 LEVEL 1 QUICK MAP

```
OOP Foundation
│
├── class → blueprint
├── object → actual instance
├── attribute → object data
├── method → object behavior
├── __init__ → initialization
└── self → current object
```

🎯 LEVEL 1 INTERVIEW POINTS

Class vs Object

```
Class = blueprint
Object = instance created from blueprint
```

`self`

`self` refers to the current instance.

`__init__`

Used to initialize instance data when an object is created.

Method

A function defined inside a class.

🟡 LEVEL 2 — INTERMEDIATE

🔄 Object State, Instance Variables, Methods, Return Values, `__str__`

---

1. Instance Variables

Variables stored separately inside each object.

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Here:

```
self.name
self.age
```

are instance variables.

Example:

```
student1 = Student("Khushi", 21)
student2 = Student("Alex", 20)
```

Conceptually:

```
student1
├── name = Khushi
└── age = 21

student2
├── name = Alex
└── age = 20
```

Each object has its own values.

⭐ **Most forgettable mistake:** Instance variables belong separately to each object; changing one object's value does not automatically change another object's value.

---

2. Instance Variable vs Local Variable

```
class Student:

    def __init__(self, name):
        self.name = name
        school = "ABC School"
```

Variable	Type	Lifetime/Location
`self.name`	Instance variable	Stored in object
`school`	Local variable	Local to `__init__`

This works:

```
student.name
```

This does not:

```
student.school
```

because `school` was not attached using `self`.

⭐ **Most forgettable mistake:** A variable becomes an object attribute only when attached using `self`.

---

3. Object State

State = current values of an object's attributes.

Example:

```
Task
├── name = Learn OOP
├── priority = High
└── status = Pending
```

After:

```
task.complete()
```

state becomes:

```
Task
├── name = Learn OOP
├── priority = High
└── status = Completed
```

---

4. Changing Object State

```
class Task:

    def __init__(self, name):
        self.name = name
        self.status = "Pending"

    def complete(self):
        self.status = "Completed"
```

Calling:

```
task.complete()
```

changes the object's state.

---

5. Methods With Parameters

Methods can receive data.

```
class BankAccount:

    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
```

Usage:

```
account = BankAccount(1000)

account.deposit(500)
```

```
print(account.balance)
```

Output:

```
1500
```

Here:

```
self.balance → object's data
amount       → method parameter
```

⭐ **Most forgettable mistake:** `amount` is temporary input; `self.balance` is stored object data.

---

6. Methods Can Return Values

```
class Calculator:

    def __init__(self, number):
        self.number = number

    def square(self):
        return self.number ** 2
```

Usage:

```
calculator = Calculator(5)

result = calculator.square()

print(result)
```

Output:

```
25
```

`print()` VS `return`

`print()`	`return`
Displays result	Sends result back
Mainly for output	Used by caller
Doesn't provide reusable result	Can be stored in variable

Example:

```
result = calculator.square()
```

⭐ **Most forgettable mistake:** `print()` displays a value, but `return` makes the value available to the caller.

---

7. Methods Can Change + Return

```
class BankAccount:

    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            return False

        self.balance -= amount
        return True
```

Usage:

```
account = BankAccount(1000)

success = account.withdraw(300)

print(success)
print(account.balance)
```

Output:

```
True
700
```

Method:

```
checks
 ↓
changes state
 ↓
returns result
```

---

8. Methods Can Call Other Methods

```
class Task:

    def __init__(self, name):
        self.name = name
        self.status = "Pending"

    def complete(self):
        self.status = "Completed"

    def is_completed(self):
```

```
        return self.status == "Completed"
```

Usage:

```
task.complete()

print(task.is_completed())
```

Output:

```
True
```

---

9. `__str__()`

`__str__()` controls the human-readable representation of an object.

Without it:

```
print(student)
```

usually gives an unhelpful object representation.

With:

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} ({self.age})"
```

Now:

```
student = Student("Khushi", 21)

print(student)
```

Output:

```
Khushi (21)
```

⭐ **Most forgettable mistake:** `__str__()` must return a string, not print one.

---

10. Real Task Class — Level 2

```
class Task:
```

```
    def __init__(self, name, priority):
        self.name = name
        self.priority = priority
        self.status = "Pending"

    def complete(self):
        self.status = "Completed"

    def is_completed(self):
        return self.status == "Completed"

    def __str__(self):
        return f"{self.name} | {self.priority} | {self.status}"
```

Usage:

```
task = Task("Learn OOP", "High")

print(task)

task.complete()

print(task)
```

Output:

```
Learn OOP | High | Pending
Learn OOP | High | Completed
```

🧠 LEVEL 2 QUICK MAP

```
Object Behavior
│
├── instance variables
├── object state
├── changing state
├── method parameters
├── return values
├── methods calling methods
└── __str__()
```

🎯 LEVEL 2 INTERVIEW POINTS

What is an instance variable?

A variable stored separately for each object, usually using `self`.

What is object state?

The current values of an object's attributes.

Why `self.name` instead of `name`?

`self.name` stores the value inside the object; `name` is just a parameter/local variable.

`return` vs `print()`?

`print()` displays; `return` sends a value back to the caller.

What is `__str__()`?

A special method used to provide a human-readable string representation of an object.

🔴 LEVEL 3 — ADVANCED

🏛️ Class Variables, Class Methods, Static Methods, Alternative Constructors

---

1. Instance vs Class Variables

```
class Student:

    school = "ABC School"

    def __init__(self, name):
        self.name = name
```

Here:

`school`

is a **class variable**.

`self.name`

is an **instance variable**.

⭐ **Most forgettable mistake:** A class variable is not the same as an instance variable; one belongs to the class, the other belongs to each object.

---

2. Main Difference

Instance Variable	Class Variable
Belongs to individual object	Belongs to class
Usually written with `self`	Defined directly in class
Different object can have different value	Shared class-level value
Example: `self.name`	Example: `school`

Conceptually:

```
Student
│
├── school = ABC School   ← class variable
│
├── student1
│    └── name = Khushi
│
└── student2
     └── name = Alex
```

---

3. Shared Class Data

```
class Student:

    school = "ABC School"

    def __init__(self, name):
        self.name = name
```

Both can access:

```
student1.school
student2.school
```

Initially:

```
ABC School
ABC School
```

Changing:

```
Student.school = "XYZ School"
```

changes the class-level value.

---

4. Attribute Shadowing

This is important.

```
student1.school = "Private School"
```

This creates an instance-level `school` for `student1`.

Conceptually:

```
Student.school
    ↓
ABC School

student1.school
    ↓
Private School

student2.school
    ↓
ABC School
```

So don't confuse:

```
Student.school
```

with:

```
student1.school
```

⭐ **Most forgettable mistake:** Assigning `student1.school` creates or changes an instance attribute; it does not necessarily change `Student.school`.

---

5. Class Variable for Counting Objects

```
class Student:

    count = 0

    def __init__(self, name):
        self.name = name
        Student.count += 1
```

Create:

```
s1 = Student("A")
s2 = Student("B")
s3 = Student("C")
```

Then:

```
print(Student.count)
```

Output:

```
3
```

Useful for class-wide information.

---

6. Class Method

A class method works with the **class**, not one specific object.

Syntax:

```
@classmethod
def method(cls):
    ...
```

`cls` refers to the current class.

Think:

```
self → current object
cls  → current class
```

⭐ **Most forgettable mistake:** A class method normally uses `cls`, not `self`.

---

7. Class Method Example

```
class Student:

    school = "ABC School"

    @classmethod
    def change_school(cls, new_school):
        cls.school = new_school
```

Call:

```
Student.change_school("XYZ School")
```

Now:

```
print(Student.school)
```

Output:

```
XYZ School
```

---

8. When to Use `@classmethod`

Use a class method when the operation concerns the **class as a whole**.

Common uses:

```
Change class-level configuration
Track class-level information
Create alternative constructors
Manage shared class data
```

---

9. Alternative Constructor

Normal constructor:

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Normal creation:

```
student = Student("Khushi", 21)
```

Suppose data arrives as:

```
Khushi,21
```

Create an alternative constructor:

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_string(cls, data):
        name, age = data.split(",")
        return cls(name, int(age))
```

Now:

```
student = Student.from_string("Khushi,21")
```

This creates a `Student` object.

Key idea

```
__init__()
    ↓
normal object creation

classmethod
    ↓
alternative way to create object
```

This pattern is highly useful when data comes from files, databases, APIs, etc.

⭐ **Most forgettable mistake:** An alternative constructor should return `cls(...)`, not just parsed data.

---

10. Static Method

A static method belongs logically to the class but doesn't need:

```
self
```

or:

```
cls
```

Syntax:

```
@staticmethod
def method(...):
    ...
```

Example:

```
class Calculator:

    @staticmethod
    def add(a, b):
        return a + b
```

Usage:

```
Calculator.add(10, 20)
```

Output:

```
30
```

---

11. Why Static Methods?

Use a static method when the operation:

- doesn't need object data
- doesn't need class data
- logically belongs to the class

Example:

```
class User:

    @staticmethod
    def is_valid_email(email):
        return "@" in email
```

Usage:

```
User.is_valid_email("abc@gmail.com")
```

⭐ **Most forgettable mistake:** A static method does not automatically receive `self` or `cls`.

---

12. Three Method Types

Method	Decorator	First parameter	Works with
Instance	None	`self`	Object
Class	`@classmethod`	`cls`	Class
Static	`@staticmethod`	None	Neither

Example:

```
class Example:

    def instance_method(self):
        pass

    @classmethod
    def class_method(cls):
        pass

    @staticmethod
    def static_method():
        pass
```

---

13. How to Decide?

Ask these questions:

Does it need object data?

```
YES → instance method
```

Example:

```
def complete(self):
```

Does it need class-level data?

```
YES → class method
```

Example:

```
@classmethod
def change_school(cls):
```

Does it need neither?

```
YES → static method
```

Example:

```
@staticmethod
def validate_email(email):
```

⭐ **Most forgettable mistake:** Choose the method type based on the data it needs: object data, class data, or neither.

---

14. Real File Automation Example

This connects directly to your future OS/File Organizer project.

```
class FileItem:

    total_files = 0

    def __init__(self, name, extension):
        self.name = name
        self.extension = extension

        FileItem.total_files += 1

    def get_filename(self):
        return self.name

    @classmethod
    def get_total_files(cls):
        return cls.total_files

    @staticmethod
    def is_image(extension):
        return extension.lower() in [".jpg", ".png", ".jpeg"]
```

Usage:

```
file1 = FileItem("photo", ".jpg")
file2 = FileItem("report", ".pdf")
```

Instance method:

```
file1.get_filename()
```

Class method:

```
FileItem.get_total_files()
```

Static method:

```
FileItem.is_image(".jpg")
```

This demonstrates all three method types in one realistic example.

🧠 LEVEL 3 QUICK MAP

```
Class-Level OOP
│
├── class variables
├── instance vs class data
├── attribute shadowing
├── @classmethod
├── cls
├── alternative constructors
└── @staticmethod
```

🎯 LEVEL 3 INTERVIEW POINTS

What is a class variable?

A variable defined at class level and shared through the class unless shadowed by an instance attribute.

`self` vs `cls`?

```
self → current instance
cls  → current class
```

What is a class method?

A method bound to the class, declared using `@classmethod`.

What is an alternative constructor?

A class method that provides another way to create an object.

What is a static method?

A method that doesn't automatically receive `self` or `cls`.

Class method vs static method?

```
classmethod → needs/accesses class
staticmethod → doesn't need class or object
```

🧠 OOP LEVEL 1–3 MASTER REVISION

```
CLASS
 ↓
Blueprint

OBJECT
 ↓
Instance created from class

ATTRIBUTE
 ↓
Data

METHOD
 ↓
Behavior

self
 ↓
Current object

__init__
 ↓
Initialize object

INSTANCE VARIABLE
 ↓
Object-specific data

STATE
 ↓
Current object data

__str__
 ↓
Human-readable object representation

CLASS VARIABLE
 ↓
Class-level/shared data

cls
 ↓
```

```
Current class

@classmethod
 ↓
Class-related operation / alternative constructor

@staticmethod
 ↓
Utility operation needing neither object nor class
```

🎤 Interview-Focused OOP Questions

These are more relevant to your goal than basic practice questions.

🟢 Beginner Interview

1. What is OOP and why is it used?
2. What is the difference between a class and an object?
3. What is an instance?
4. What is an attribute?
5. What is a method?
6. What is `self` in Python?
7. Why is `self` explicitly written in Python methods?
8. What is `__init__()`?
9. Is `__init__()` a constructor in the strict technical sense?
10. What happens when `ClassName()` is executed?

🟡 Intermediate Interview

11. What is an instance variable?
12. What is object state?
13. Difference between local variable and instance variable?
14. Difference between `print()` and `return` in a method?
15. Can a method modify object state?
16. Can one method call another method?
17. What is `__str__()`?
18. What happens if `__str__()` doesn't return a string?
19. How are multiple objects of the same class different?
20. How would you design a `Task` class for a real To-Do application?

🔴 Advanced Interview

21. What is a class variable?
22. Instance variable vs class variable?
23. What happens when an instance and class have attributes with the same name?
24. What is attribute shadowing?
25. What is `@classmethod`?

26. What is `cls`?
27. When should you use a class method?
28. What is an alternative constructor?
29. Why are class methods useful when loading data from files/APIs?
30. What is `@staticmethod`?
31. When should a static method be used?
32. Difference between instance method, class method and static method?
33. Why shouldn't everything be made a static method?
34. How would you design a `FileItem` class for a file-organizing application?
35. How would you track the number of objects created using a class variable?
36. How would you create an object from CSV/file data using a class method?
37. How would you decide whether functionality belongs inside an object method or a standalone function?
38. Explain `self`, `cls`, and no first parameter using one real-world example.
39. Design a class representing a bank account and explain its state and behavior.
40. Design classes for a Smart File Organizer and explain which data should belong to objects versus the class.

⚠️ Important Interview Rule

Don't memorize definitions only.

If asked:

"What is a class method?"

A stronger answer is:

```
A class method is a method bound to the class rather than a
particular instance. It receives cls as its first parameter and
is useful for class-level operations and alternative constructors.
```

Then give:

```
@classmethod
def from_string(cls, data):
    ...
```

That demonstrates **understanding + syntax + practical use**.

📘 OOP QUICK REVISION NOTES

LEVEL 4 — ADVANCED OOP

Inheritance, `super()`, Method Overriding, Polymorphism, Composition, Encapsulation & Duck Typing

Goal: Quick revision for exams + interviews while retaining the practical understanding and examples from the detailed lessons.

---

1. INHERITANCE

Simple meaning

Inheritance allows a child class to reuse attributes and methods of a parent class.

```
Parent
  ↓
Child
```

Example:

```
class Animal:

    def eat(self):
        print("Animal is eating")


class Dog(Animal):
    pass


dog = Dog()
dog.eat()
```

Output:

```
Animal is eating
```

`Dog` automatically gets `eat()` from `Animal`.

Important terminology

Term	Meaning
Parent class	Base class / Superclass
Child class	Derived class / Subclass
`class Dog(Animal)`	Dog inherits Animal

Real-world relationship

```
Animal
├── Dog
├── Cat
```

```
└── Bird
```

Because:

```
Dog IS-A Animal
Cat IS-A Animal
```

---

2. WHY INHERITANCE?

Without inheritance:

```
class PDFFile:
    def __init__(self, name, size):
        self.name = name
        self.size = size


class ImageFile:
    def __init__(self, name, size):
        self.name = name
        self.size = size
```

Repeated code.

With inheritance:

```
class File:

    def __init__(self, name, size):
        self.name = name
        self.size = size


class PDFFile(File):
    pass


class ImageFile(File):
    pass
```

Now common functionality exists in one place.

---

3. CHILD CAN ADD ITS OWN FEATURES

```
class Animal:

    def eat(self):
        print("Eating")
```

```
class Dog(Animal):

    def bark(self):
        print("Woof")
dog = Dog()

dog.eat()    # inherited
dog.bark()   # own method
Dog
├── eat()   ← inherited
└── bark()  ← own
```

---

4. `super()`

Simple meaning

`super()` allows a child class to use functionality from its parent according to the inheritance hierarchy/MRO.

Example:

```
class Animal:

    def __init__(self, name):
        self.name = name


class Dog(Animal):

    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Instead of repeating:

```
self.name = name
```

the child delegates parent initialization to:

```
super().__init__(name)
```

Mental model

```
Dog.__init__()
      ↓
super().__init__()
      ↓
Animal.__init__()
```

---

5. `super()` WITH METHODS

```
class Animal:

    def speak(self):
        print("Animal sound")


class Dog(Animal):

    def speak(self):
        super().speak()
        print("Woof")
dog = Dog()
dog.speak()
```

Output:

```
Animal sound
Woof
```

Professional use

Use `super()` when a child wants to **extend** parent behavior instead of completely replacing it.

---

6. METHOD OVERRIDING

Meaning

A child class provides its own implementation of a method already defined by the parent.

```
class Animal:

    def speak(self):
        print("Animal sound")


class Dog(Animal):

    def speak(self):
        print("Woof")
Dog().speak()
```

Output:

```
Woof
```

The child implementation replaces the inherited implementation for that object.

Quick distinction

Concept	Meaning
Inheritance	Get functionality from parent
Overriding	Replace/modify inherited behavior
`super()`	Reuse parent behavior

---

7. POLYMORPHISM

Simple meaning

Same interface/method → different behavior.

Example:

```
class Dog:

    def speak(self):
        print("Woof")


class Cat:

    def speak(self):
        print("Meow")
animals = [Dog(), Cat()]

for animal in animals:
    animal.speak()
```

Output:

```
Woof
Meow
```

The code only knows:

```
animal.speak()
```

It doesn't need to know the exact class.

---

8. POLYMORPHISM WITH INHERITANCE

```
class Animal:
```

```
    def speak(self):
        print("Animal sound")


class Dog(Animal):

    def speak(self):
        print("Woof")


class Cat(Animal):

    def speak(self):
        print("Meow")
animals = [Dog(), Cat(), Animal()]

for animal in animals:
    animal.speak()
```

Output:

```
Woof
Meow
Animal sound
```

Professional benefit

New classes can be added without rewriting code that already works with the common interface.

---

9. IS-A vs HAS-A

This is **extremely important for interviews and design**.

Relationship	Meaning	Example	Usually use
IS-A	One thing is a type of another	Dog IS-A Animal	Inheritance
HAS-A	One object contains/uses another	Car HAS-A Engine	Composition

IS-A

```
class Dog(Animal):
    pass
```

HAS-A

```
class Car:

    def __init__(self):
        self.engine = Engine()
```

---

10. COMPOSITION

Meaning

Composition means one object **contains or uses another object**.

```
class Engine:

    def start(self):
        print("Engine started")


class Car:

    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()
        print("Car started")
car = Car()
car.start()
```

Output:

```
Engine started
Car started
```

Relationship:

```
Car
└── Engine
```

Professional rule

Prefer composition when the relationship is **HAS-A**, rather than forcing inheritance.

---

11. INHERITANCE vs COMPOSITION

Inheritance	Composition
IS-A	HAS-A
Strong relationship	Flexible relationship
Reuses parent behavior	Delegates work to another object
`class Dog(Animal)`	`self.engine = Engine()`
Can create tight coupling	Often more flexible

Example for your project

Bad:

```
class FileOrganizer(File):
    pass
```

Why?

```
FileOrganizer IS NOT-A File
```

Better:

```
class FileOrganizer:

    def __init__(self, scanner, mover):
        self.scanner = scanner
        self.mover = mover
```

---

12. ENCAPSULATION

Meaning

Encapsulation means keeping an object's internal state controlled and exposing a clean interface.

Example:

```
class BankAccount:

    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
```

Instead of allowing uncontrolled changes:

```
account._balance = -50000
```

the class provides controlled operations.

---

13. `_variable` vs `__variable`

Syntax	Meaning
`name`	Public by convention
`_name`	Internal/protected-by-convention
`__name`	Name mangling; stronger protection from accidental access

Example:

```
class BankAccount:

    def __init__(self):
        self._balance = 1000
        self.__pin = 1234
```

Important

Python does **not** provide absolute private variables like some languages.

`__name` uses **name mangling**.

---

14. `@property`

Allows a method to behave like an attribute.

```
class BankAccount:

    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance
```

Now:

```
account.balance
```

instead of:

```
account.balance()
```

---

15. PROPERTY SETTER

```
class Person:

    def __init__(self, age):
        self.age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):

        if value < 0:
```

```
            raise ValueError("Age cannot be negative")

        self._age = value
```

Now:

```
person.age = -5
```

is rejected.

Why useful?

```
property
   ↓
controlled access
   ↓
validation
   ↓
valid object state
```

---

16. DUCK TYPING

Python's philosophy:

If an object provides the behavior I need, I can use it.

```
class Dog:

    def speak(self):
        print("Woof")


class Robot:

    def speak(self):
        print("Beep")
things = [Dog(), Robot()]

for thing in things:
    thing.speak()
```

Output:

```
Woof
Beep
```

Neither class needs to inherit from the same parent.

Key idea

```
Don't ask:
"What exact type are you?"
```

```
Ask:
"Can you perform the operation I need?"
```

⭐ LEVEL 4 INTERVIEW CHEAT SHEET

Question	Short answer
What is inheritance?	Mechanism for deriving a class from another class
What is overriding?	Child provides a different implementation of inherited method
What is `super()`?	Used to continue/reuse parent/hierarchy behavior
What is polymorphism?	Same interface, different implementations
What is composition?	Object contains/uses another object
IS-A means?	Inheritance relationship
HAS-A means?	Composition relationship
What is encapsulation?	Controlling access to internal state/implementation
What is `_x`?	Internal-use convention
What is `__x`?	Name-mangled attribute
What is `@property`?	Attribute-like controlled access to a method
What is duck typing?	Behavior/interface matters more than exact type

---

🎯 LEVEL 4 — MOST EXPECTED INTERVIEW QUESTIONS

Conceptual

1. What is inheritance in Python?
2. What are the advantages and disadvantages of inheritance?
3. What is method overriding?
4. Difference between overloading and overriding?
5. Does Python support method overloading in the traditional sense?
6. What is polymorphism?
7. Explain polymorphism with a real-world example.
8. Difference between inheritance and composition?
9. What is the difference between IS-A and HAS-A?
10. Why is composition often preferred over deep inheritance?
11. What is encapsulation?
12. Is Python truly private?
13. Difference between `_name` and `__name`?
14. What is name mangling?
15. What is `@property`?

16. What is duck typing?
17. What is the difference between polymorphism through inheritance and duck typing?

Code/interview problems

18. Predict the output of an overriding example.
19. Explain the execution order when `super()` is used.
20. Design `Animal → Dog/Cat` using polymorphism.
21. Design `File → PDF/Image/Video` using inheritance.
22. Decide whether a given relationship should use inheritance or composition.
23. Design a `BankAccount` with controlled balance modification.
24. Refactor a class hierarchy that has excessive inheritance.
25. Design a file-processing system where every file implements `process()`.

LEVEL 5 — EXPERT / PROFESSIONAL OOP

Abstract Classes, Interfaces, Multiple Inheritance, MRO, Dunder Methods, Dataclasses, Dependency Injection & SOLID

---

1. ABSTRACT CLASSES

Simple meaning

An abstract class is a **blueprint/contract** that defines what child classes must provide.

```
from abc import ABC, abstractmethod


class File(ABC):

    @abstractmethod
    def open(self):
        pass
```

You cannot normally instantiate it:

```
File()
```

because `open()` is abstract.

---

2. IMPLEMENTING AN ABSTRACT CLASS

```
class PDFFile(File):

    def open(self):
        print("Opening PDF")
```

Now:

```
pdf = PDFFile()
pdf.open()
```

Output:

```
Opening PDF
```

Mental model

```
Abstract class
      ↓
defines contract
      ↓
child implements contract
      ↓
object can be created
```

---

3. MULTIPLE ABSTRACT METHODS

```
class File(ABC):

    @abstractmethod
    def open(self):
        pass

    @abstractmethod
    def close(self):
        pass
```

Child must implement both:

```
class PDFFile(File):

    def open(self):
        print("Open PDF")

    def close(self):
        print("Close PDF")
```

---

4. INTERFACE-STYLE DESIGN

Python doesn't have a separate `interface` keyword like Java/C#.

Abstract classes can provide interface-like contracts.

```
class Storage(ABC):

    @abstractmethod
    def save(self, data):
        pass

    @abstractmethod
    def load(self):
        pass
```

Implementations:

```
Storage
├── FileStorage
├── DatabaseStorage
└── CloudStorage
```

Each must provide:

```
save()
load()
```

---

5. ABSTRACT CLASS + POLYMORPHISM

```
storages = [
    FileStorage(),
    DatabaseStorage()
]

for storage in storages:
    storage.save("hello")
```

Same:

```
storage.save()
```

Different implementations.

```
Abstraction
     +
Polymorphism
     ↓
Flexible architecture
```

---

6. MULTIPLE INHERITANCE

Python allows a class to inherit from multiple classes.

```
class Flyer:

    def fly(self):
        print("Flying")


class Swimmer:

    def swim(self):
        print("Swimming")


class Duck(Flyer, Swimmer):
    pass
duck = Duck()

duck.fly()
duck.swim()
```

Output:

```
Flying
Swimming
```

---

7. MRO — METHOD RESOLUTION ORDER

MRO means:

The order Python follows when searching for methods in an inheritance hierarchy.

```
class A:

    def hello(self):
        print("A")


class B:

    def hello(self):
        print("B")


class C(A, B):
    pass
C().hello()
```

Output:

```
A
```

Because `A` comes before `B`.

Inspect it:

```
print(C.mro())
```

Conceptually:

```
C → A → B → object
```

---

8. `super()` + MRO

```
class A:

    def hello(self):
        print("A")


class B(A):

    def hello(self):
        print("B")
        super().hello()


class C(B):

    def hello(self):
        print("C")
        super().hello()
C().hello()
```

Output:

```
C
B
A
```

Professional understanding

Don't think of `super()` only as:

"call my direct parent."

Think:

Continue the method lookup according to MRO.

---

9. DUNDER METHODS

Dunder = **double underscore** methods.

Examples:

```
__init__
__str__
__repr__
__len__
__eq__
__add__
__contains__
```

They allow objects to interact naturally with Python's syntax and built-in functions.

---

10. `__str__`

Human-friendly representation.

```
class Student:

    def __init__(self, name):
        self.name = name

    def __str__(self):
        return self.name
print(Student("Alex"))
```

Output:

```
Alex
```

---

11. `__repr__`

Developer/debug-oriented representation.

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Student(name={self.name!r}, age={self.age!r})"
```

Useful for debugging and inspecting objects.

Quick distinction

Method	Purpose
`__str__`	Human-friendly
`__repr__`	Developer/debug-friendly

---

12. `__len__`

Controls:

```
len(object)
```

Example:

```
class Cart:

    def __init__(self):
        self.items = []

    def __len__(self):
        return len(self.items)
```

Now:

```
len(cart)
```

internally uses:

```
cart.__len__()
```

---

13. `__contains__`

Controls:

```
item in object
class Cart:

    def __init__(self):
        self.items = []

    def __contains__(self, item):
        return item in self.items
```

Now:

```
"Laptop" in cart
```

works naturally.

---

14. `__eq__`

Controls equality:

```
object1 == object2
```

Example:

```
class User:

    def __init__(self, username):
        self.username = username

    def __eq__(self, other):
        return self.username == other.username
```

Now two users with the same username can compare equal according to your defined logic.

---

15. OPERATOR OVERLOADING

Python allows objects to define operators.

Examples:

```
+  → __add__
-  → __sub__
*  → __mul__
== → __eq__
<  → __lt__
```

Example:

```
class Money:

    def __init__(self, amount):
        self.amount = amount

    def __add__(self, other):
        return Money(self.amount + other.amount)
```

Now:

```
m3 = m1 + m2
```

uses:

```
m1.__add__(m2)
```

---

16. DATACLASSES

For classes primarily used to hold structured data:

```
from dataclasses import dataclass


@dataclass
class Student:
    name: str
    age: int
    grade: str
```

Then:

```
student = Student("Alex", 20, "A")
```

Dataclasses automatically provide useful functionality such as initialization and representation.

Good use cases

```
FileInfo
User
Product
Task
Configuration
API data
Settings
```

---

17. DATACLASS DEFAULT VALUES

```
@dataclass
class Task:
    name: str
    priority: str = "Medium"
    completed: bool = False
task = Task("Learn OOP")
```

Conceptually:

```
name      → Learn OOP
priority  → Medium
completed → False
```

---

18. DEPENDENCY INJECTION

Bad/tightly coupled

```
class FileOrganizer:

    def __init__(self):
        self.storage = FileStorage()
```

The class decides exactly which implementation to use.

Better

```
class FileOrganizer:

    def __init__(self, storage):
        self.storage = storage
```

Then:

```
organizer = FileOrganizer(FileStorage())
```

or:

```
organizer = FileOrganizer(DatabaseStorage())
```

Meaning

Give an object the dependencies it needs instead of making it create them internally.

---

19. WHY DEPENDENCY INJECTION?

It provides:

```
Flexibility
Testability
Loose coupling
Easy replacement
Better architecture
```

Example:

```
FileOrganizer
      ↓
   Storage
   /     \
File    Database
```

The organizer doesn't need to know the concrete storage implementation.

---

20. SOLID PRINCIPLES

Letter	Principle	Quick meaning
S	Single Responsibility	One main responsibility
O	Open/Closed	Extend without unnecessarily modifying existing code
L	Liskov Substitution	Child should be safely usable where parent is expected
I	Interface Segregation	Don't force unnecessary interfaces/methods
D	Dependency Inversion	Depend on abstractions, not concrete implementations

---

21. S — SINGLE RESPONSIBILITY

Bad:

```
class FileOrganizer:

    def scan(self):
        ...

    def move(self):
        ...

    def send_email(self):
        ...

    def generate_report(self):
        ...
```

Too many responsibilities.

Better:

```
FileScanner
FileMover
EmailSender
ReportGenerator
```

Rule

One class should have one clear reason to change.

---

22. O — OPEN/CLOSED

The system should be:

```
Open for extension
Closed for unnecessary modification
```

Example:

```
File
├── PDF
├── Image
├── Video
└── Audio
```

Adding `AudioFile` should ideally not require rewriting the whole processing system.

Polymorphism helps:

```
for file in files:
    file.process()
```

---

23. L — LISKOV SUBSTITUTION

Simple test:

Can I substitute the child wherever the parent is expected without breaking expected behavior?

If:

```
Dog IS-A Animal
```

then a `Dog` should behave in a way consistent with the contract expected from `Animal`.

A subclass that fundamentally breaks parent expectations is a design warning.

---

24. I — INTERFACE SEGREGATION

Don't force a class to implement methods it doesn't need.

Instead of one huge interface:

```
Machine
├── print()
├── scan()
└── fax()
```

prefer smaller contracts:

```
Printable
Scannable
Faxable
```

Classes implement only what they actually support.

---

25. D — DEPENDENCY INVERSION

Avoid:

```
class Organizer:

    def __init__(self):
        self.storage = FileStorage()
```

Prefer:

```
class Organizer:

    def __init__(self, storage):
        self.storage = storage
```

The high-level class depends on an abstraction/contract rather than being tightly coupled to one implementation.

---

26. PROFESSIONAL OOP ARCHITECTURE

A real project might look like:

```
project/
│
├── models/
│   ├── file.py
│   └── task.py
│
├── services/
│   ├── scanner.py
│   ├── classifier.py
│   ├── mover.py
│   └── organizer.py
│
├── storage/
│   ├── base.py
│   └── file_storage.py
│
├── utils/
│   └── validators.py
│
└── main.py
```

The important idea isn't the exact folder names.

It's **separation of responsibilities**.

⭐ LEVEL 5 INTERVIEW CHEAT SHEET

Topic	Interview answer
Abstract class	Blueprint/contract that can contain abstract methods
`ABC`	Base class used to create abstract classes
`@abstractmethod`	Marks a method that subclasses must implement
Interface in Python	Usually represented through ABCs/protocol-style design
Multiple inheritance	Class inherits from multiple base classes
MRO	Order Python uses to resolve methods
`super()`	Continues/reuses behavior according to MRO
Dunder methods	Special methods recognized by Python
`__str__`	User-friendly representation
`__repr__`	Developer/debug representation
`__len__`	Defines `len(obj)`
`__eq__`	Defines equality behavior
Operator overloading	Giving operators custom object behavior
Dataclass	Concise way to create data-focused classes
Dependency injection	Supplying dependencies externally
SOLID	Five principles for maintainable OOP design

🎯 LEVEL 5 — MOST EXPECTED INTERVIEW QUESTIONS

Advanced concepts

1. What is an abstract class?
2. Why would you use `ABC` and `@abstractmethod`?
3. Can an abstract class contain normal methods?
4. What is an interface in Python?
5. How can ABCs be used to implement interface-like design?
6. What is multiple inheritance?
7. What is MRO?
8. How do you inspect a class's MRO?
9. How does `super()` work with multiple inheritance?
10. What is the difference between `super()` and directly calling the parent class?
11. What are dunder methods?
12. Difference between `__str__` and `__repr__`?
13. What is operator overloading?

14. How does `len(obj)` work internally?
15. What is a dataclass?
16. When should you use a dataclass instead of a normal class?
17. What is dependency injection?
18. Why does dependency injection improve testing?
19. What is loose coupling?
20. What is tight coupling?

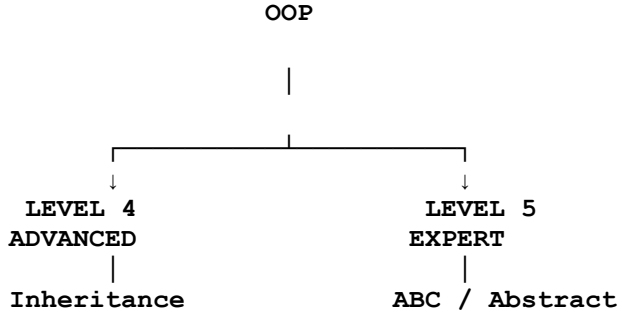
SOLID interview questions

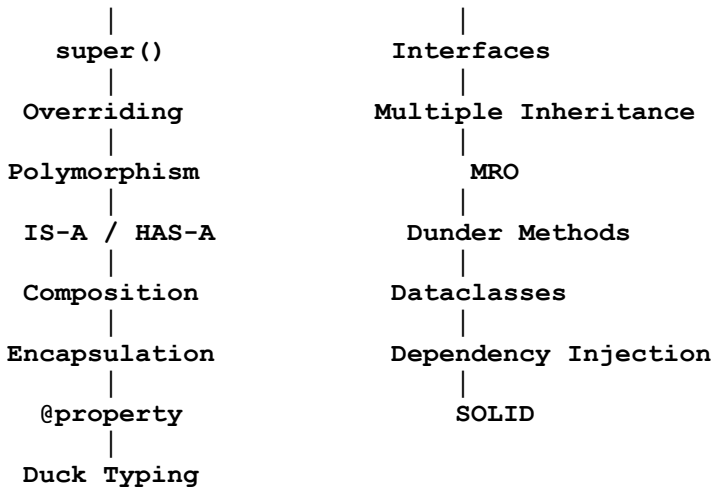
21. Explain all five SOLID principles.
22. Explain Single Responsibility with a real project example.
23. Explain Open/Closed with polymorphism.
24. Explain Liskov Substitution with an inheritance example.
25. Explain Interface Segregation.
26. Explain Dependency Inversion.
27. How does dependency injection relate to Dependency Inversion?
28. Why can excessive inheritance be dangerous?
29. Why is composition often preferred over inheritance?
30. Design a maintainable File Organizer using SOLID principles.

Real-project interview questions

31. Design a file-processing system supporting PDF, image, video and audio files.
32. How would you add a new file type without modifying existing processing code?
33. How would you make storage interchangeable between file, database and cloud storage?
34. How would you unit-test a `FileOrganizer` without actually moving files?
35. How would you separate scanning, classification, moving and logging responsibilities?
36. Where would you use an abstract class in your File Organizer?
37. Where would you use composition?
38. Where would you use polymorphism?
39. Where would dependency injection be useful?
40. Explain the architecture of your File Organizer and justify each class.

🧠 ONE-PAGE OOP LEVEL 4 + 5 MAP




super()
Overriding
Polymorphism
IS-A / HAS-A
Composition
Encapsulation
@property
Duck Typing
Interfaces
Multiple Inheritance
MRO
Dunder Methods
Dataclasses
Dependency Injection
SOLID